

Introduction to User Interface: High Level and Low Level UI

Mr. Tan Chiang Kang



User Interface

- In mobile applications, user interface can be roughly classified into two major categories:
 - High Level UI
 - Consists of many components that can be directly used with some capability to customise the components for regular usage of input and output.
 - Low Level UI
 - Allows design in the raw state to any sort of custom design for functionality beyond the High level UI can offer.

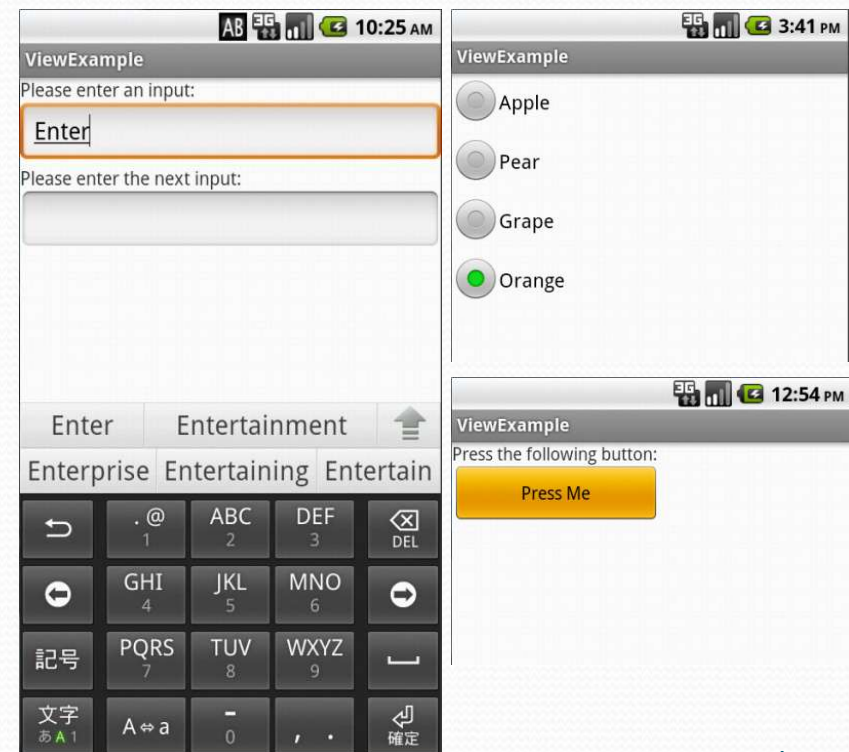
High Level UI

- A high level UI allows the developers to make use of predefined UI components
- These components are usually commonly used in many different applications (e.g. textboxes, checklists, buttons, etc)
- The developers usually get limited control on how the components are designed or displayed
- However, developers can easily build a user interface by putting different UI components in place

High Level UI Components

- High level UI usually provides some commonly used UI components (known as Views in Android), for example:

- Text display (TextView)
- Input textboxes (EditText)
- Dropdown list (Spinner)
- Button (Button)
- Checkboxes (CheckBox)
- Radiobuttons (RadioButton)



Other UI Components

- There are more UI components, including those that are less commonly used or those that are only used for specific applications, for example:
 - Date-time pickers
 - Auto-complete input boxes
 - Maps
 - Galleries
 - Tab sheets
 - Etc

Grouping Up Components

- To work with multiple UI components, we need to place them together into a single holder
- These holders are usually invisible, they help put smaller UI components in place (in Android, they are known as ViewGroup)
- The common holders to hold UI components are Layouts
 - Layouts can be used to define how the components are to be placed

Layouts in Anroid

- Layouts in Android:
 - Linear Layout
 - Relative Layout
 - Table Layout
 - Grid Layout
 - Constraint Layout

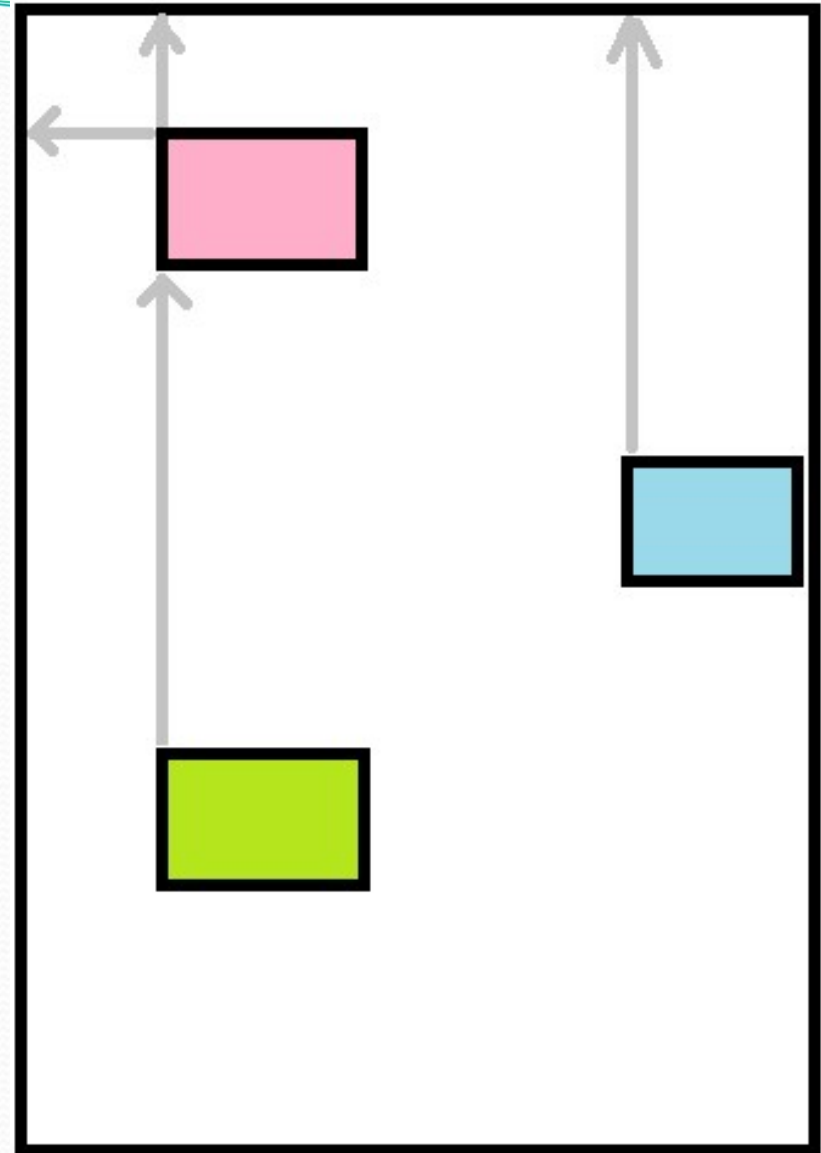
LinearLayout

- Aligns each child View in either vertical column or horizontal row
- A vertical layout has rows of Views, while a horizontal layout has columns of Views
- The Linear Layout manager enables you to specify a “weight” for each child View that controls the relative size of each within the available space



RelativeLayout

- The most flexible of the native layouts, the RelativeLayout lets you define the positions of each child View relative to the others and to the screen boundaries



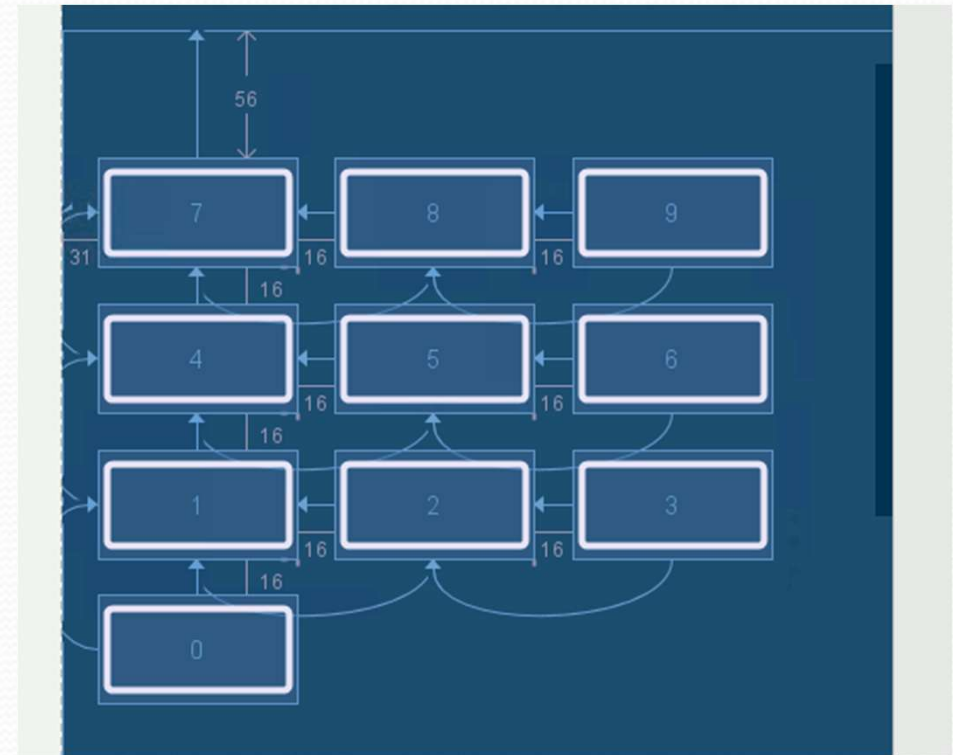
TableLayout

- Allows you to lay out Views using a grid of rows and columns
- Tables can span multiple rows and columns, and columns can be set to shrink or grow



ConstraintLayout

- A new layout is introduced to try and reduce the complexity
- It behaves like RelativeLayout but also partially like other layouts
- Hence, many modern layouts can be accomplished via this single type



Optimizing Layouts

- Inflating layouts into Activities is an expensive process
- Use of additional nested layout could impact the performance
- To optimise the performance, it is recommended to:
 - **Avoid unnecessary nesting:** only use nesting when it is necessary, some nesting may have same result as with no nesting
 - **Avoid using too many Views:** never include more than 80 Views in a layout
 - **Avoid deep nesting:** avoid any nesting more than 10 levels deep

Designing UI in Action

- When designing UI for mobile applications, usually two major design methods were available:
 - Drag and drop – elements are dragged and dropped into UI-based design panel, which is usually XML based
 - Hardcoding - each element is created using programming codes
- Which one is better?

UI Design Using XML Layout

- Modern mobile application's UI design is usually done through the design of an XML-based layout, often through drag and drop
- The XML-based design is simple and easy to manage, and reduces development time for developers
- However, it can only be used on static UI (predefined and non changing UI)

Layout (XML)

- In Android, typically your UI is defined in an XML file, which may look like this:

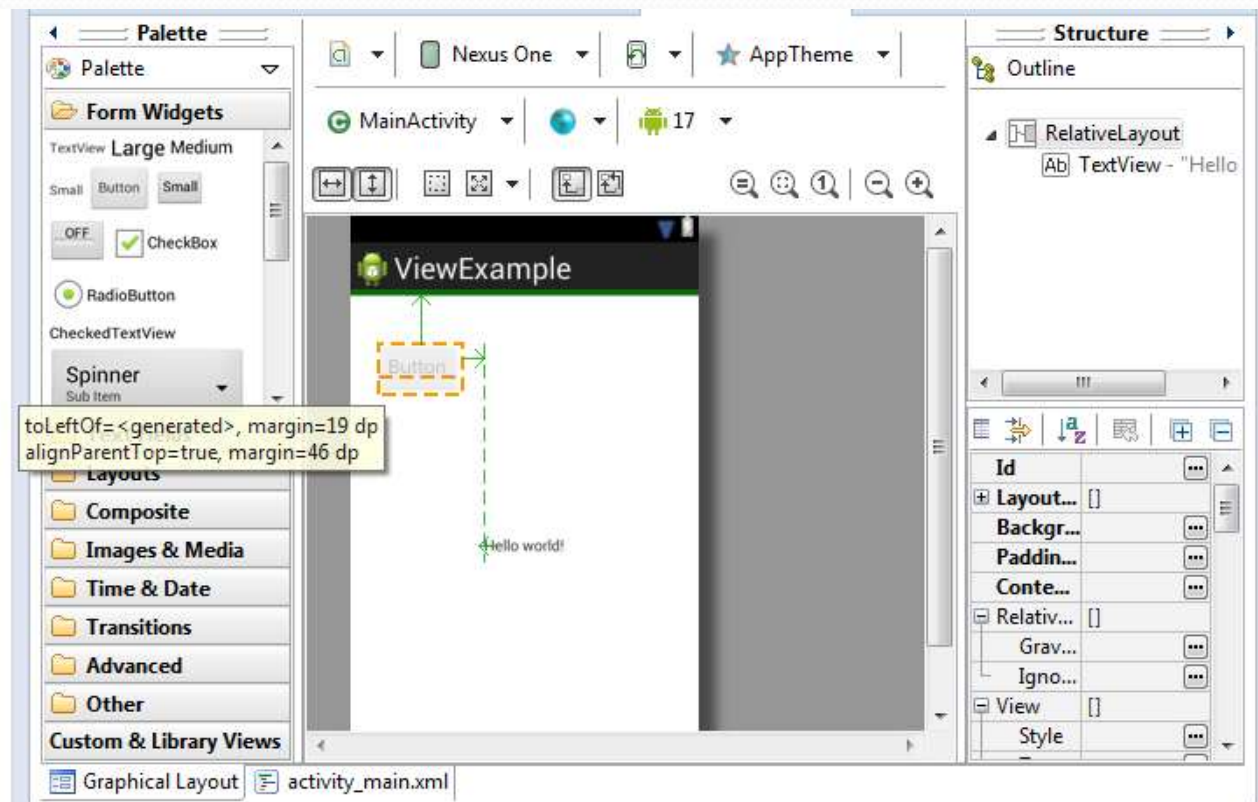
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello" />

</LinearLayout>
```

Drag and Drop UI Tools

- Eclipse IDE also allows drag and drop to create the UI elements



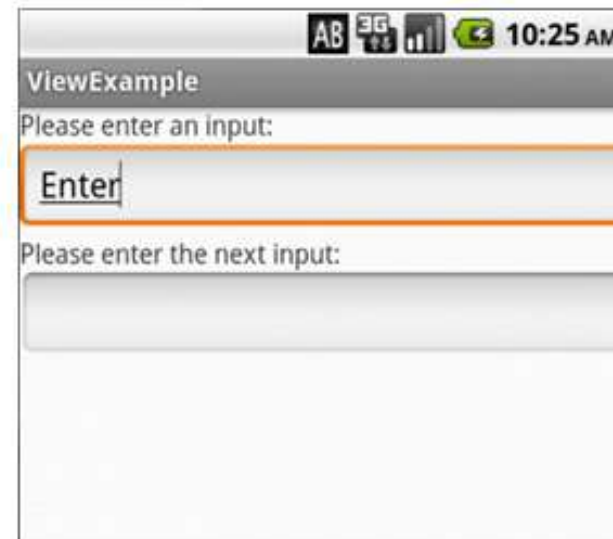
UI Design by Coding

- Developers can also hardcode a UI without using an XML layout design
- The developers may need to spend more time to properly construct objects and manage them
- This process is more time consuming, but can be done during runtime
- This allows the developers to design dynamic UI

Example

```
TextView text = new TextView(this);  
text.setText("Please enter an input:");  
EditText edit = new EditText(this);
```

```
LinearLayout ll = new LinearLayout(this);  
ll.setOrientation(LinearLayout.VERTICAL);  
ll.addView(text);  
ll.addView(edit);  
setContentView(ll);
```



Nesting Layouts

- Layouts can be nested below one another, and this is called nesting layouts

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent" >
```

```
    <RelativeLayout
        android:layout_width="wrap_content"
        android:layout_height="match_parent" >
```

```
</RelativeLayout>
```

```
    <LinearLayout
        android:layout_width="match_content"
        android:layout_height="match_parent" >
```

```
</LinearLayout>
```

```
</LinearLayout>
```

XML-based Managements

- In the XML layouts, one may notice that the text was referring to yet another reference point instead of the actual text itself

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/hello" />
</LinearLayout>
```

XML-based Managements

- The reference point was actually stored in another XML for strings, i.e. `res/values/strings.xml` (Android uses many different XML to manage different parts and resources of the application)
- This practice is often used in modern mobile application design
- It does provide its own advantage. Can you think of what they are?

XML-based Managements

- Internationalisation with Android XML strings
- Android automatically selects the appropriate string resource based on the user's device language settings
- Locale specific directories to be created
 - For each additional language, you create a new values directory with a language-specific qualifier (e.g., values-es for Spanish, values-fr for French)
- When the application runs, the Android system checks the device's current locale.
 - If the device is set to French, Android loads string values from `res/values-fr/strings.xml`

Handling User Events

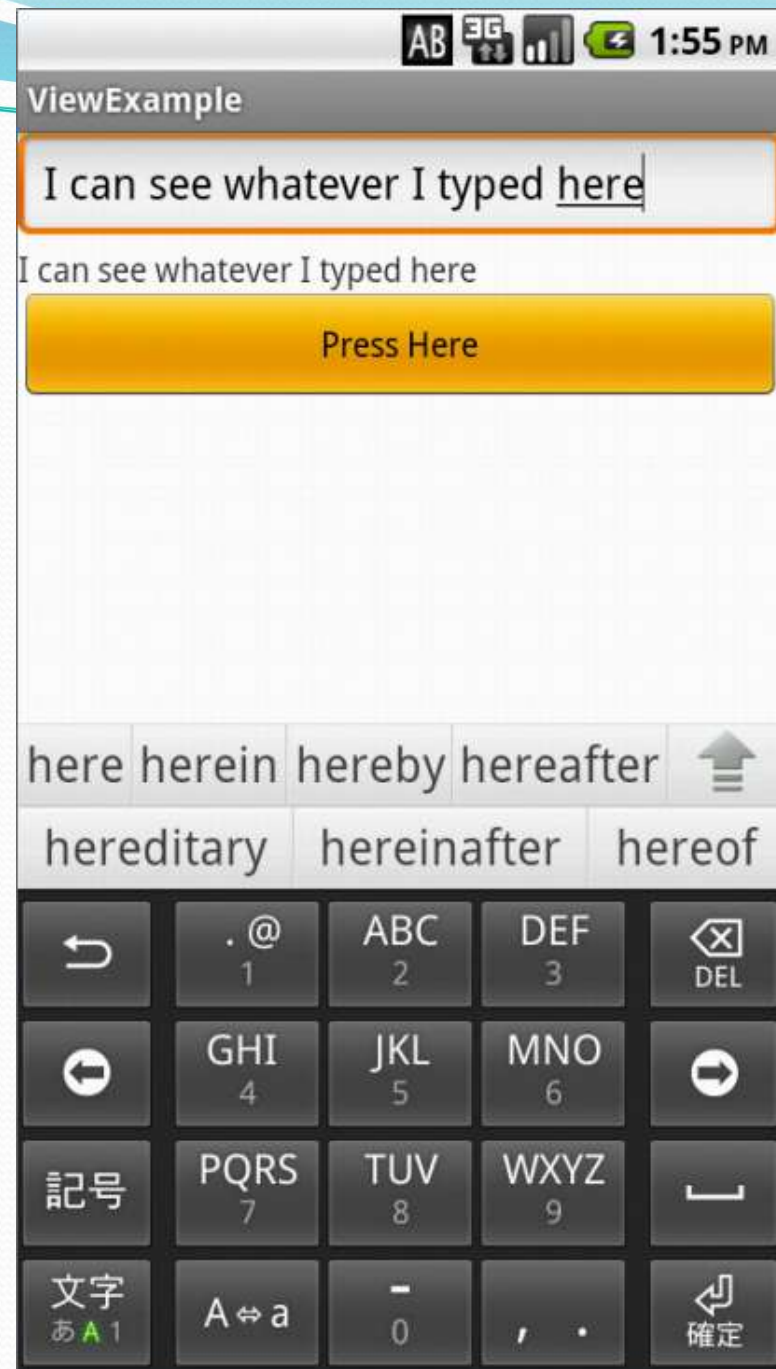
- After displaying different UI components that allows user to give an input, the system has to react to it and produce some outcome
- This is achieved by first listening to these specific user events (a click, a drag or etc) and react accordingly when the user events are triggered
- The listeners are specific to different UI components according to their own behaviour

Button Click

- The commonest trigger used is the buttons, which is usually used to confirm on entered or selected data
- A listener is usually set on buttons to listen for click events
- When a click is triggered, the corresponding methods will be executed
- Each button usually has a distinct listener, but some buttons can also share a listener

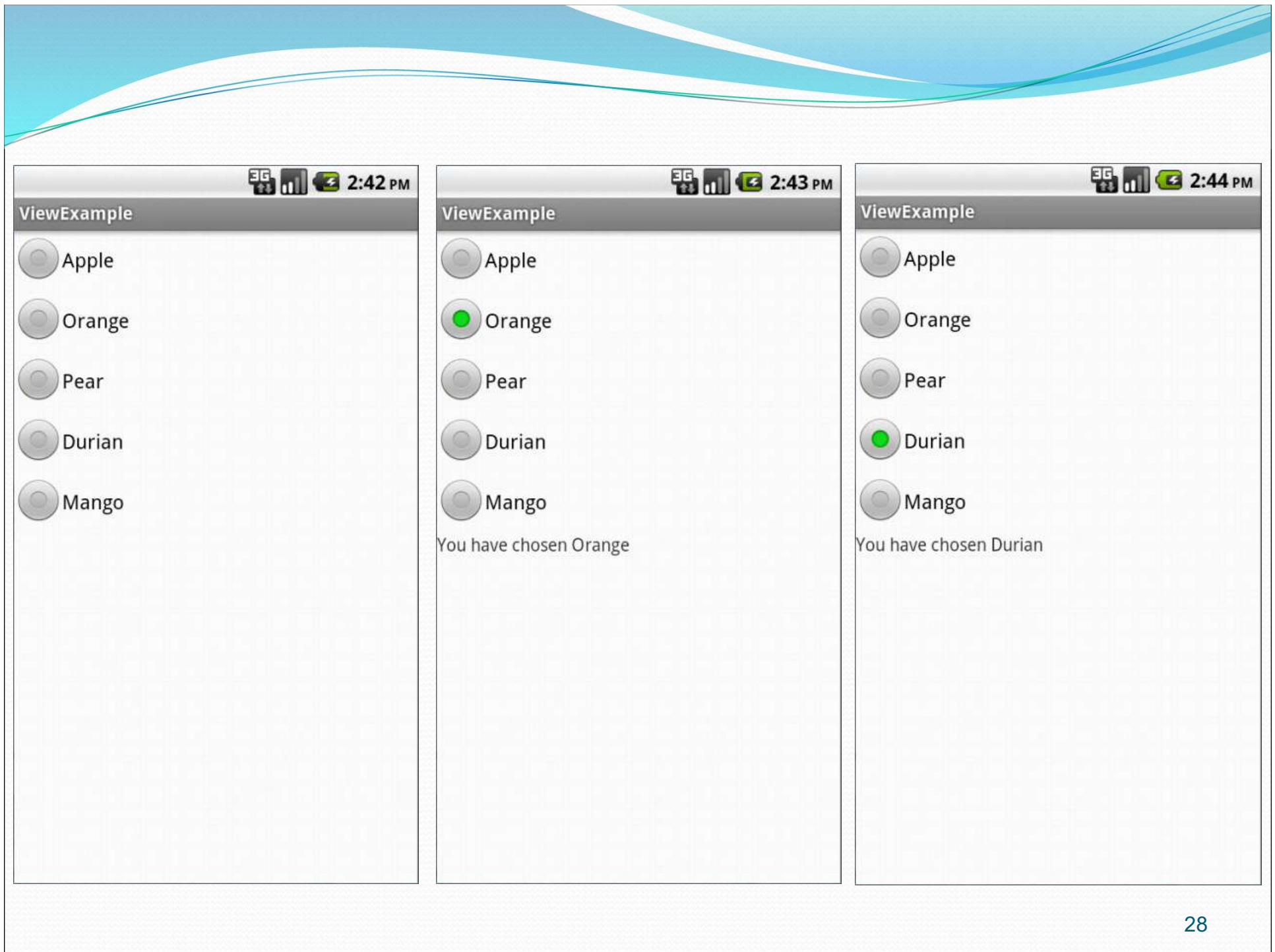
Button Click

- It can be used to work with other components that do not have the capability to respond by itself
- The button click will be a helpful way to confirm that the entry of data or selection of choices is completed
- Usually a button is used to confirm the entry of textbox, selection of a checkbox group and radiobutton group



Changes on UI Components

- Sometimes, we do not only react to a confirmed and completed form due to a button click
- We often need to update the UI from time to time to reflect changes that were done on some UI components
- Changes of states of some UI components can often also be listened
- For example, while we were typing, while we were selecting some choices, etc



Other User Events

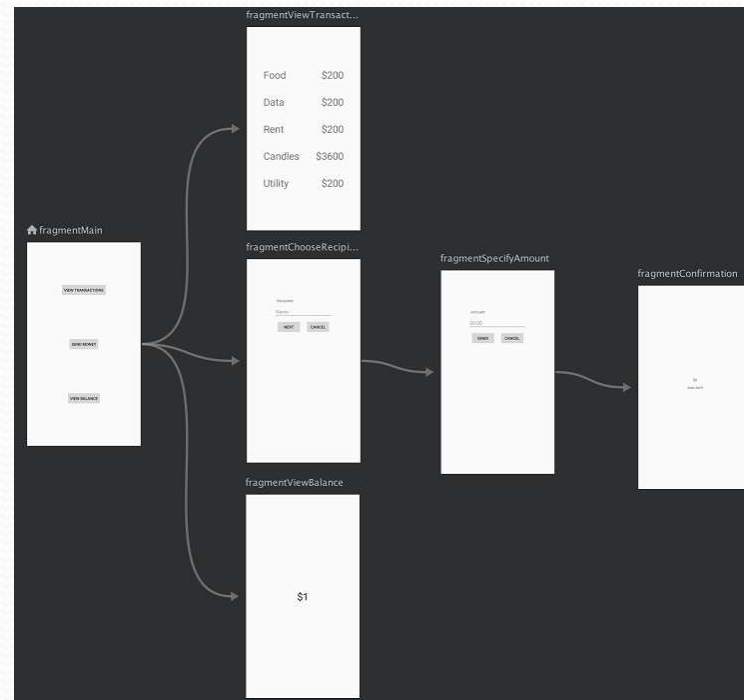
- A number of other handlers are available to handle different events
- They are generally similar but use different listeners and actions
- For example:
 - Listening for long clicks
 - Listening for gestures
 - Listening for screen orientation changes
 - Listening for focus changes

Connecting the Pages

- In mobile applications, to reduce scroll-based navigations, many applications are designed into multiple smaller pages
- They may proceed from one page to another and the way of communicating between the pages generally differ with different technologies

Connecting the Pages

- Android Studio now offers Navigation Components to link up different Activities, Fragments or other types.
- iOS offers similar option even earlier which is known as Storyboard

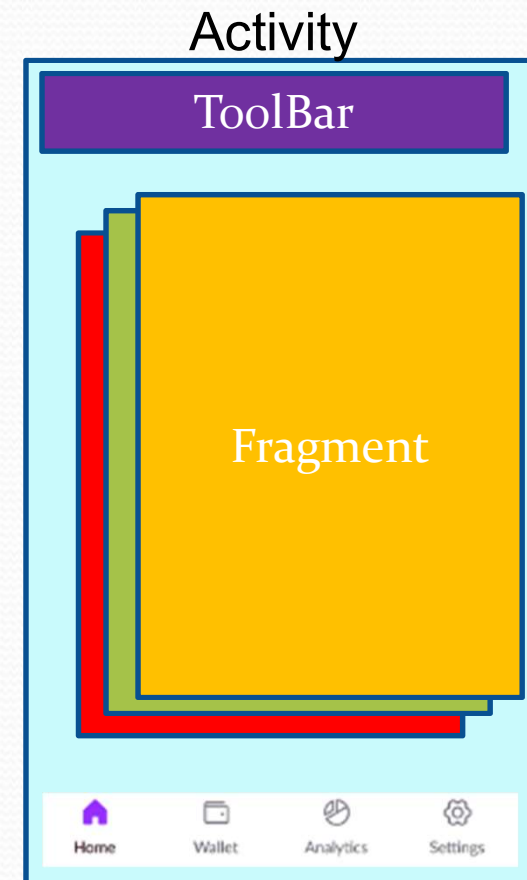


Using Intents in Android

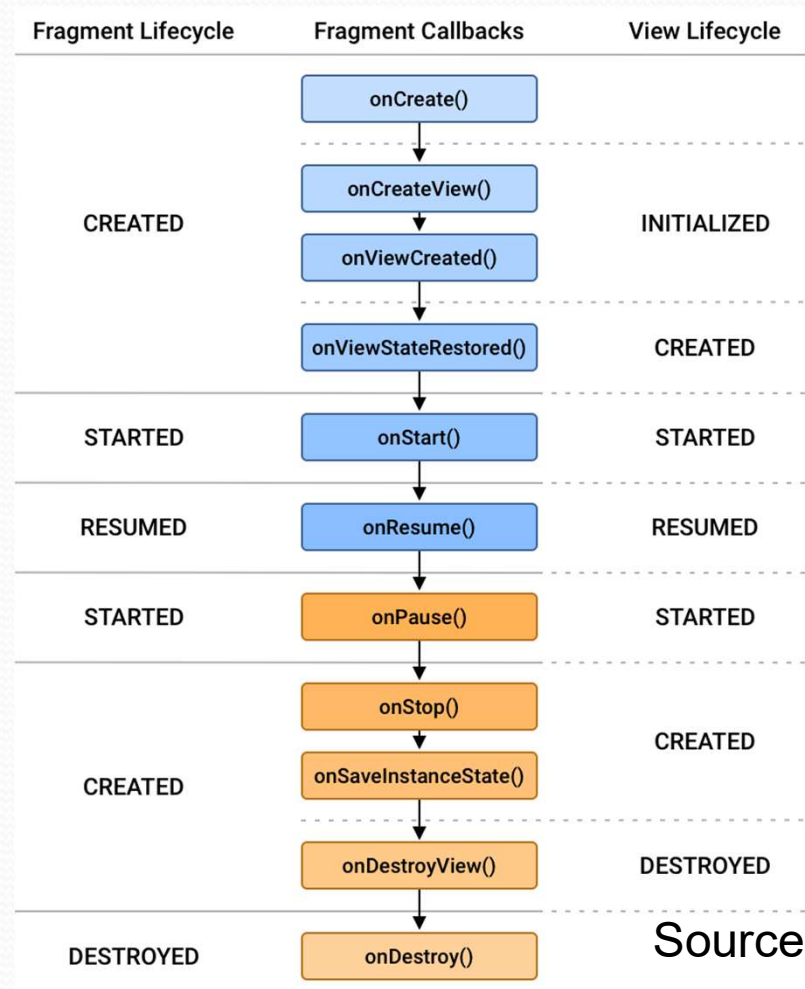
- In Android, **Intents** can be used
- Intents can be used for data-passing or navigation that works both within application or between applications
- Intent can be used to switch the focus from the current Activity to the target Activity
- The target Activity will execute its content which may show a different layout

Fragments

- You do not need to create multiple independent Activities when they are closely related to one another
- You can create multiple Fragments that group up in the same Activity
- The Fragments must always be hosted in an Activity
- Each Fragment will have its own lifecycle, but which will also be affected by the host Activity lifecycle



Fragment Lifecycle



Source: developer.android.com

Fragments Navigation

- Jetpack Navigation Component is the recommended way to navigate between Fragments
- Navigation graph (nav_graph.xml) is used to link to destination Fragment
- The central component NavController is then triggered to carry out the navigation
- Data passing is done using Safe Args
- Current Fragment automatically added to back stack

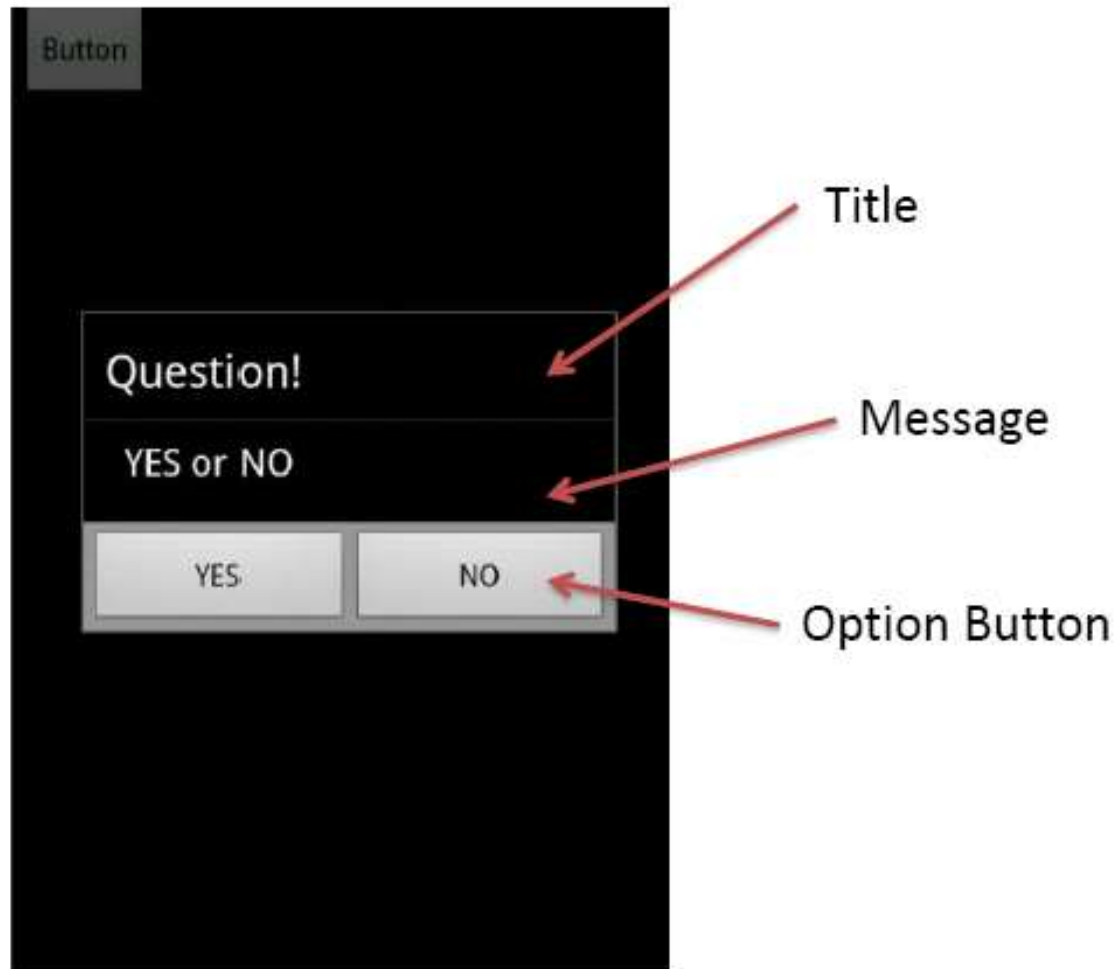
Other Useful Tools

- Some other tools are useful to enhance the mobile applications, such as:
 - Dialog Box
 - Option Menu
 - Context Menu

Dialog Box

- Dialog box is a box that appears on a display screen to present information or request input
- Typically, dialog boxes are temporary - they disappear once you have entered the requested information
- Dialog boxes can be used to display temporary messages (e.g. errors)
- It can also be used to take some information from the user without disturbing the current UI

Dialog Box (AlertDialog)

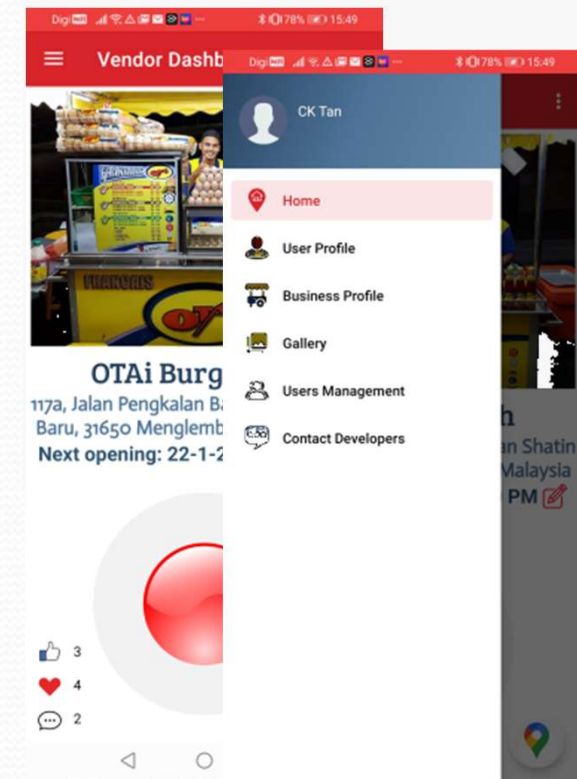
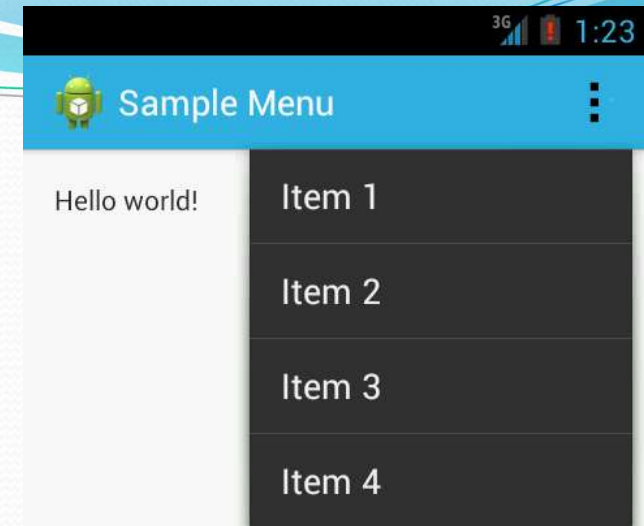


Extra Popup Menus

- Due to the nature of restricted size in the display, some information is usually hidden from the user when it is not required
- Menus for example, are usually hidden from the user
- Menus are often used to contain options that would only be required for certain scenarios
- In mobile applications, 2 types of menus can be used, a general menu (e.g. options menu) and a component-specific menu (context menu)

General Menu

- Known as “Options Menu” in Android
- Triggered via a button on the top right corner
- The menus can be added to the activity to provide help, navigation, additional controls or configuring options
- Drawers are a popular alternative used for general menus today



Component-Specific Menu

- Known as “Context Menu” in Android
- Such menu is triggered by example long clicking on specific UI components
- It provides a contextual menu to the user where additional actions can be performed on selected item
- This allows the user to select choices that were only relevant to that exact item itself



Low Level UI

- Sometimes the available High Level components cannot be used to fulfill what is needed
- Canvas can be used to create custom made components from scratch at low level
- This can help to produce drawings and some animations



phillipmartin.info

Custom View

- To use Low Level UI, the main aim is to create a custom component
- In Android, a custom View can be created to be a subclass of View
- We can paint the canvas by defining the behaviour of the pixels, by drawing shapes, filling in colors, displaying texts, images and etc
- We can create any UI components that we prefer, for any purpose, including those existing in High Level UI

Canvas

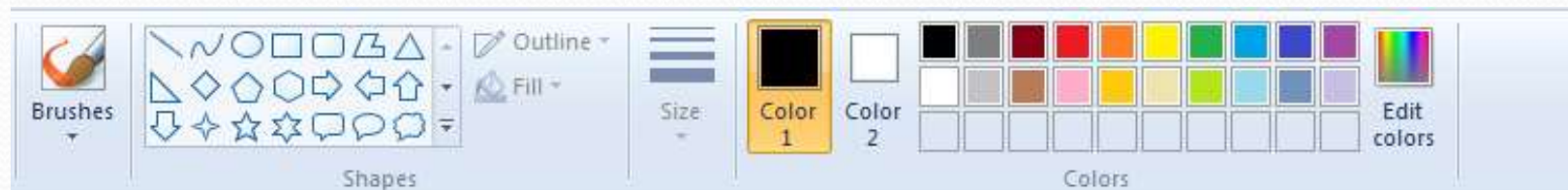
- The Canvas is bound by container view in rectangular region
- The width and height of the screen can be retrieved, which is useful because the resolution of the screen greatly varies from one device to another
- The obtained width and height can be used to work out a relative location within the screen to draw something

How to Design Canvas?

- The idea of Low Level UI is to design some UI components from scratch
- Therefore, should we design it pixel by pixel?
- Imagine the number of pixels we have on our smartphones nowadays, will it not be tiring to design it pixel by pixel?
- Luckily we don't need to, many basic elements are often made available in designing low level UI as well, such as basic shapes, texts and also the use of existing images

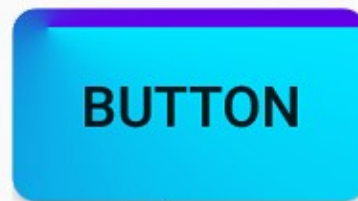
Paint Settings

- Before drawing anything, the developers need to select the paint details and styles
- They include colours, size, font and etc, with some characteristics applied to certain contents only
- Once a style is set, all drawing methods called below will behave to adopt the same style unless another is set

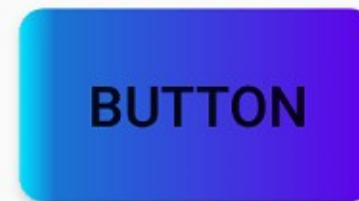


Colour Settings

- For example you can set a drawable resource with a particular colour and style (gradient, gradient angle etc), and then apply it to a button



sweep



linear



radial

Drawing Text

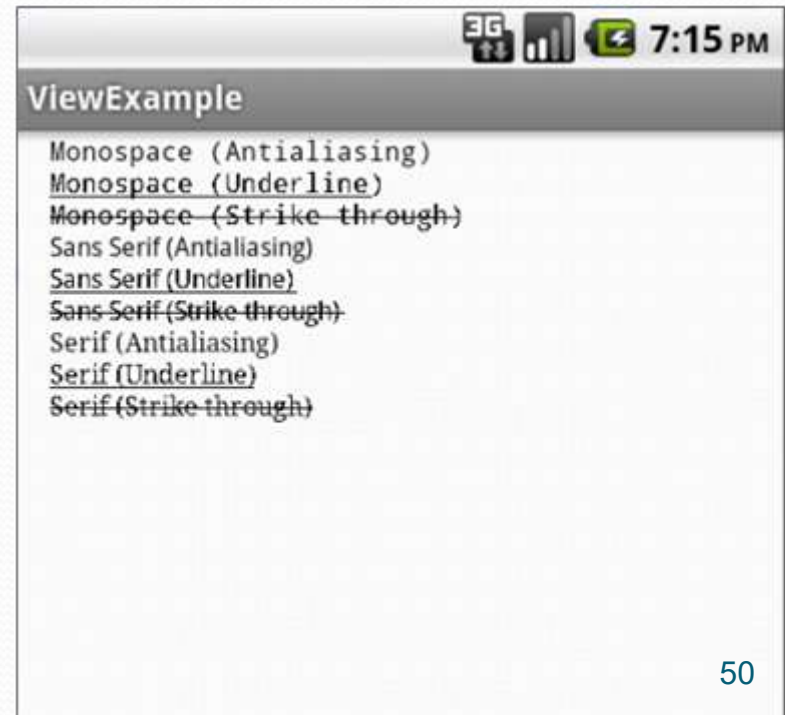
- We can use the drawing capability to draw text instead of showing them using High Level UI
- The flexibility in controlling the styles and where to show the text is stronger
- The user can choose the font, style, size and locations while painting certain text
- However, usually the choices of font and style available are often very restricted

Fonts and Typefaces

- For example in Android:
 - There are only 3 types of fonts (Typefaces) available, i.e. **Sans Serif**, **Monospace** and **Serif**
 - There are only 3 styles, i.e. **Normal**, **Bold** and **Italic**, but includes composite types
 - Besides that, you can set certain properties of typeface such as antialiasing, underlining and strike-through
- The type of fonts you can produce from such choices are quite limited

Example

- Example below shows what kind of fonts and styles you can write



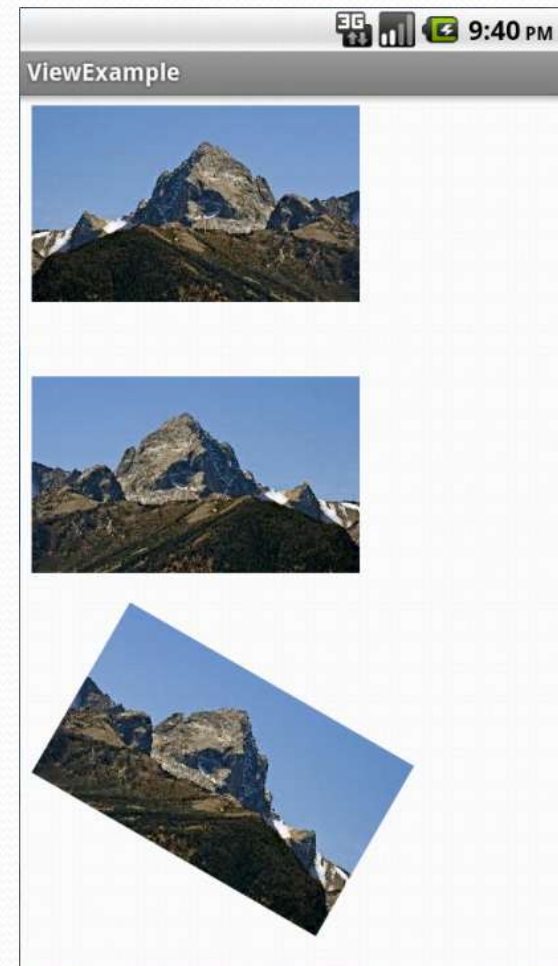
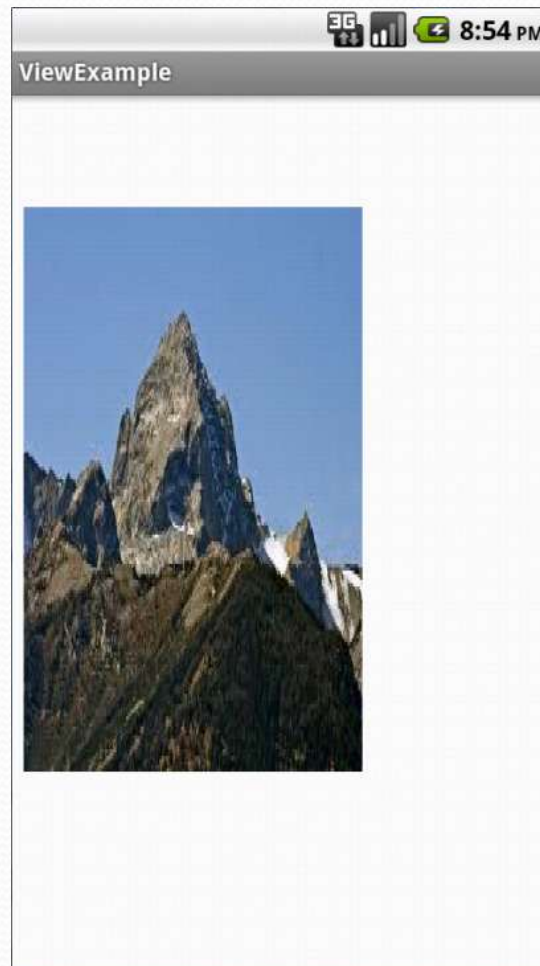
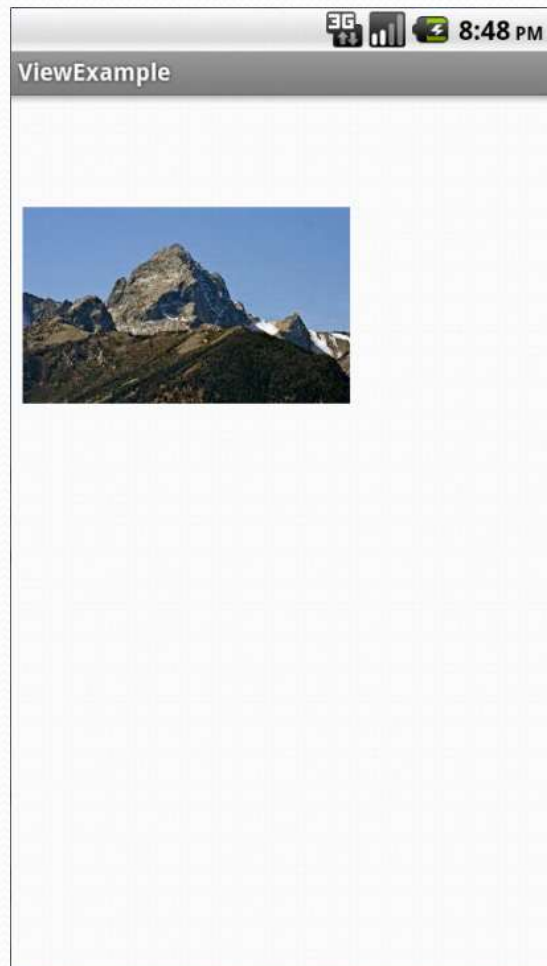
Dealing with Images

- You can always import existing images to be used in Canvas design, which is quite a popular practice
- Nicer and complicated designs can be achieved easily in advanced image processing tools than in codes
- In Android, different resolutions are made available to optimise the resources for different screens (**ldpi/mdpi/hdpi/xhdpi/xxhdpi**)
- One of the more popular portable file types would be **PNG (Portable Network Graphics)**

More with Images

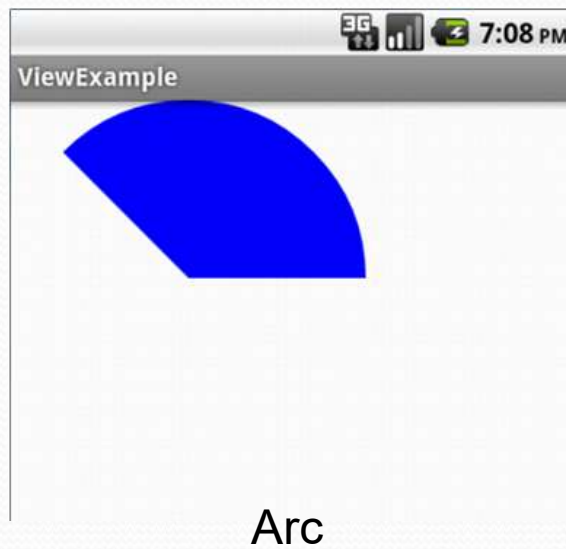
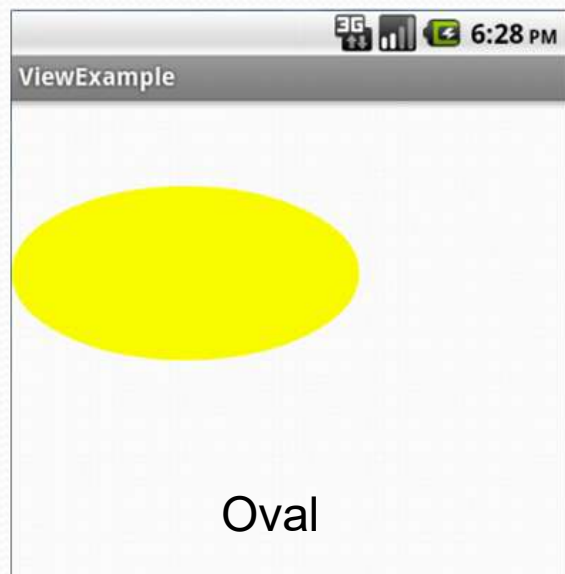
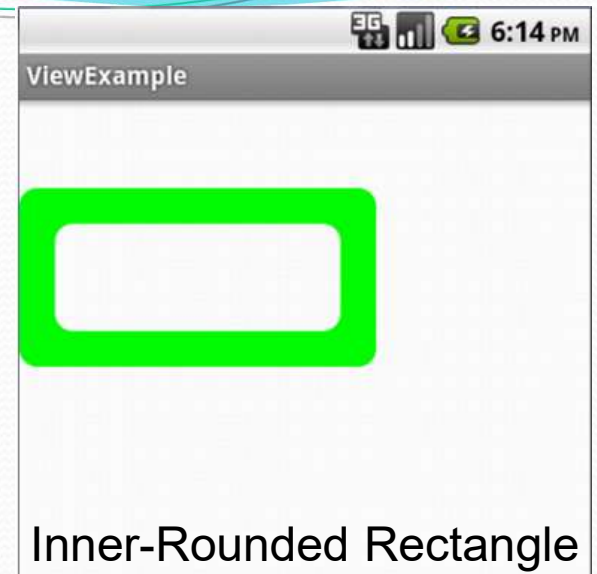
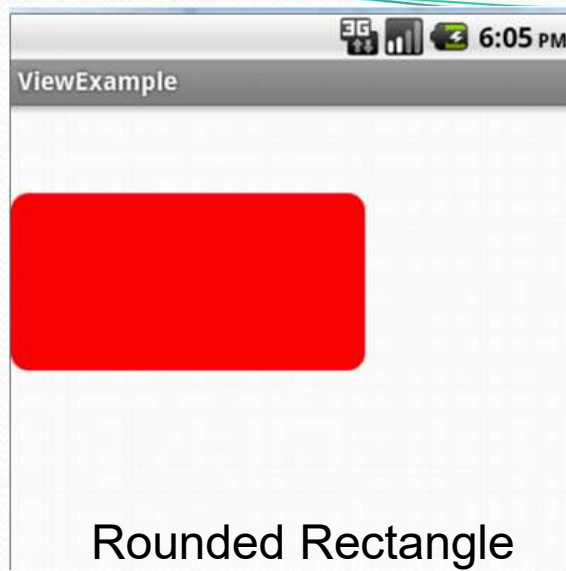
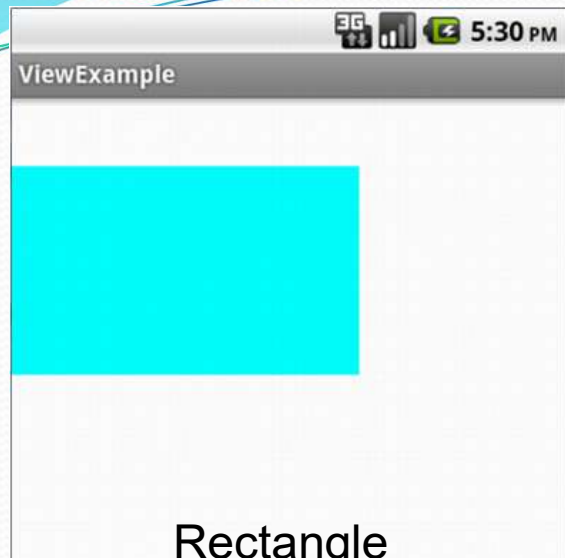
- Other than just using original images, images can be cropped, rescaled and transformed to be used as well
- Some common geometrical transformations include mirroring, rotation, translation and etc
- These open up many possibilities to how an image can be manipulated
- Often these techniques are used to create very simple games
- E.g. instead of designing very complicated movements, the avatars may just slide forward, and reflected to the other side when they change direction)

Examples



Working with Shapes

- If you are not going after complicated graphics, many custom components can be created using basic shapes
- Basic shapes that are used for drawings often include: squares, rectangles, circles, ellipses
- Android offers some of these:
 - Rectangles
 - Rounded Rectangles
 - Ovals
 - Arcs and Lines
 - Paths

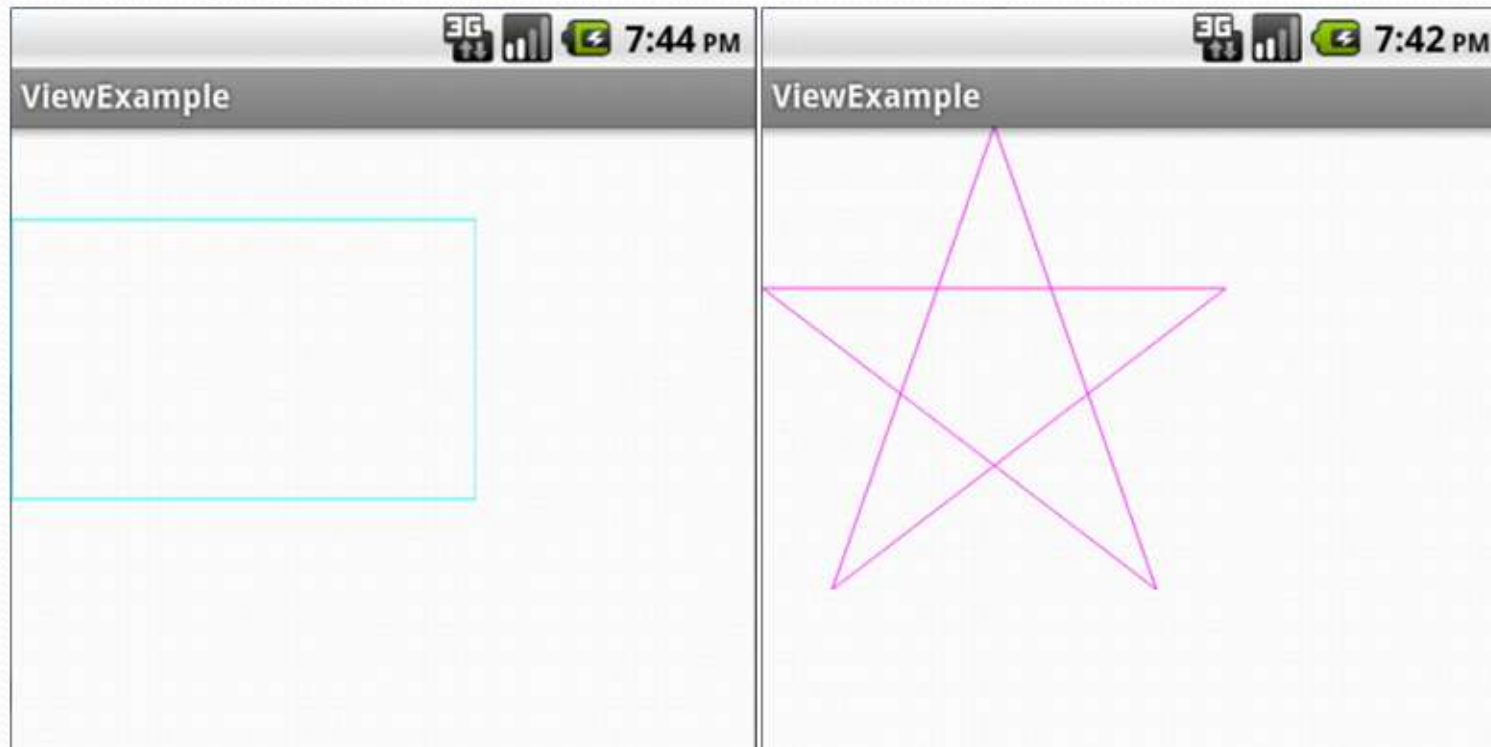


Shapes in Android

- The drawn shapes can also be regarded as Drawable object and be displayed as an image
- In High Level UI, Drawables can be displayed using **ImageView**
- This allows the developers to easily use some shapes in the High Level UI as well
- However the shapes that are allowed to be drawn as a drawable is very basic compared to those done in the Canvas

Drawing Outlines

- Besides creating filled up shapes, outlined shapes are usually made available as well, based on the filled up shapes themselves



Handling Events

- Because these components are created from scratch, their corresponding events should also be designed from scratch
- For Low Level UI, multi-touch actions are often supported as well
- The developers can program the corresponding response based on click or gesture of the user on certain part of the screen
- More efforts are required compared to High Level UI



What's Next?

- Keys to UI Design